

SIMATS ENGINEERING
Department of Computer Science &
Engineering ASSESSMENT TOOL-1 EXPERIMENTS

Course Code:	CSA1221	Course Title:	Computer Architecture
CO Assessed:	C03	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	25
C03Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	R. MANOJ KUMAR NAIK	Reg.No.:	192524436
Department:	AI&DS	Date:	12-08-2026

ANNEXURE A

EXPERIMENT 1: Register Transfer & ALU Operations

Aim

To design and simulate a digital system that performs register transfer operations and basic arithmetic/logic operations (addition, subtraction, AND, OR) using multiple registers, and to analyze the sequence of microoperations and the role of the ALU in processing data.

Tools Used

- Logisim Evolution / Digital Works (logic simulator)
- C / Python (software-level register-transfer simulation)

Theory

A digital computer consists of a set of registers (R0, R1, R2, R3 ...) connected through common data paths to an Arithmetic and Logic Unit (ALU). Data movement between registers is expressed using Register Transfer Language (RTL), e.g. $R1 \leftarrow R2$ denotes that the content of R2 is transferred to R1. The ALU accepts operands from source registers, performs the operation specified by the control signals, and places the result in a destination register. A control unit generates timing and select signals that decide which registers act as source/destination and which ALU operation is enabled for a given clock cycle.

Algorithm

- Step 1: Initialize registers R0, R1, R2 and R3 with input data values.
- Step 2: Select two source registers as ALU operands based on the control signal.
- Step 3: Decode the operation code (ADD, SUB, AND, OR) using the control unit.
- Step 4: Route the operand values to the ALU inputs through the common bus.
- Step 5: Perform the selected operation inside the ALU.
- Step 6: Transfer the ALU result back to the destination register on the next clock pulse.
- Step 7: Repeat steps 2–6 for each instruction in the operation sequence.

- Step 8: Display the final content of all registers.

Observation Table

Operation	R0	R1	R2	Result (R3)
ADD (R0+R1)	12	5	8	17
SUB (R0-R2)	12	5	8	4
AND (R0&R1)	12	5	8	4
OR (R0 R2)	12	5	8	12

Result / Analysis

The register transfer system was successfully simulated. Each micro-operation was completed within one clock cycle using the common bus to move data between registers and the ALU. It was observed that the ALU acts as the central computing element — every arithmetic or logic instruction is broken down into (i) a transfer of operands to the ALU inputs and (ii) a transfer of the ALU output to a destination register, confirming that register transfer operations form the fundamental building block of instruction execution in a processor.

EXPERIMENT 2: Single-Bus and Multi-Bus Organization

Aim

To model a computer system using single-bus and multi-bus organization, perform data transfer between registers, memory and I/O, and compare the time taken and efficiency of the two approaches.

Tools Used

- Logisim Evolution (bus and multiplexer modelling)
- Python (cycle-count simulation of bus transfers)

Theory

In a single-bus organization, all registers, the ALU, memory and I/O units share one common bus, so only one data transfer can occur at any instant; simultaneous transfers must be serialized, increasing the number of clock cycles needed for complex operations. In a multi-bus organization (e.g. a 3-bus structure with two source buses and one destination bus), multiple operands can be read from different registers and the ALU result written back

in the same cycle, allowing several transfers to proceed in parallel and reducing overall execution time at the cost of additional hardware (extra bus lines and drivers).

Algorithm

- Step 1: Model registers, ALU, memory and I/O as modules connected to the bus(es).
- Step 2: For single-bus design, allow only one source-to-destination transfer per clock cycle.
- Step 3: For multi-bus design, allow two operand reads (bus A, bus B) and one result write (bus C) in the same cycle.
- Step 4: Execute a fixed instruction sequence (e.g. $R1 \leftarrow \text{Mem}$, $R2 \leftarrow \text{Mem}$, $R3 \leftarrow R1 + R2$, $\text{Mem} \leftarrow R3$) on both models.
- Step 5: Count the number of clock cycles taken by each organization.
- Step 6: Compare total cycles and compute the percentage improvement of the multi-bus design.

Observation Table

Organization	Buses	Cycles Taken	Parallel Transfers
Single-Bus	1	4	No
Multi-Bus	3	2	Yes (2 reads + 1 write/cycle)

Result / Analysis

The single-bus model required 4 clock cycles to complete the instruction sequence since every transfer had to wait for the bus to be free. The multi-bus model completed the same sequence in 2 cycles by reading both operands simultaneously on separate buses and writing the ALU result on a third bus, giving roughly a 50% reduction in execution time. This confirms that multiple buses reduce bottlenecks and improve parallel data transfer, though they require more control logic and wiring than a single-bus design.

EXPERIMENT 3: 5-Stage Instruction Pipeline

Aim

To simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back), measure speedup and throughput, and compare its performance with a non-pipelined system.

Tools Used

- Python (pipeline timing-diagram simulator)

- MARS / Ripes (MIPS pipeline visual simulator) — optional

Theory

Pipelining overlaps the execution of multiple instructions by dividing instruction processing into independent stages — Fetch (IF), Decode (ID), Execute (EX), Memory access (MEM) and Write-back (WB). While one instruction is being decoded, the next can be fetched, so that in steady state one instruction completes every clock cycle instead of every 5 cycles as in a non-pipelined design. For n instructions and k stages, a non-pipelined processor takes $n \times k$ cycles, whereas an ideal pipeline takes $k + (n - 1)$ cycles. Speedup is defined as (nonpipelined time)/(pipelined time), and throughput is the number of instructions completed per cycle.

Algorithm

- Step 1: Define $k = 5$ pipeline stages and n instructions to execute.
- Step 2: For the non-pipelined model, compute total cycles = $n \times k$.
- Step 3: For the pipelined model, compute total cycles = $k + (n - 1)$, assuming no stalls.
- Step 4: Build a stage-vs-cycle timing diagram showing each instruction moving through IF→ID→EX→MEM→WB.
- Step 5: Compute Speedup = non-pipelined cycles / pipelined cycles.
- Step 6: Compute Throughput = n / pipelined cycles (instructions per cycle).

Timing Diagram ($n = 5$ instructions, $k = 5$ stages)

Instr	C1	C2	C3	C4	C5	C6	C7	C8	C9
I1	IF	ID	EX	MEM	WB				
I2		IF	ID	EX	MEM	WB			
I3			IF	ID	EX	MEM	WB		
Instr	C1	C2	C3	C4	C5	C6	C7	C8	C9
I4				IF	ID	EX	MEM	WB	
I5					IF	ID	EX	MEM	WB

Result / Analysis

For 5 instructions, the non-pipelined processor requires 25 cycles (5×5), while the pipelined processor completes execution in only 9 cycles ($5 + 4$), giving a speedup of about 2.78 and a throughput approaching 1 instruction per cycle in steady state. This demonstrates that pipelining significantly improves processor performance by

overlapping instruction execution, although the achievable speedup is always less than the number of stages due to pipeline fill and drain time.

EXPERIMENT 4: Pipeline Hazards and Resolution Techniques

Aim

To develop a pipeline execution scenario that exhibits data, control and structural hazards, observe their impact on instruction execution, and implement stalling, data forwarding and branch prediction to resolve them.

Tools Used

- Python (hazard-detection and stall simulator)
- Ripes / MARS (visual pipeline simulator) — optional

Theory

- Structural Hazard: occurs when two instructions need the same hardware resource (e.g. a single memory port) in the same cycle.
- Data Hazard: occurs when an instruction depends on the result of a previous instruction that has not yet completed (e.g. Read-After-Write).
- Control Hazard: occurs when the pipeline fetches instructions before a branch outcome is known, potentially fetching the wrong instructions.

These hazards are resolved using: Stalling / pipeline bubbles (inserting idle cycles until the hazard clears), Data Forwarding / Bypassing (routing the ALU result directly to a dependent instruction before it is written back), and Branch Prediction (guessing the branch outcome to avoid stalling, with a flush penalty on misprediction).

Algorithm

- Step 1: Take an instruction sequence containing a data dependency, e.g. ADD R1,R2,R3 followed by SUB R4,R1,R5.
- Step 2: Detect the RAW hazard by comparing source registers of each instruction with destination registers of the two preceding instructions.
- Step 3: Without forwarding — insert stall (bubble) cycles until the dependent value is written back to the register file.
- Step 4: With forwarding — bypass the ALU output of the producing instruction directly to the EX stage of the consuming instruction, eliminating most stalls.
- Step 5: For a branch instruction, predict the outcome (e.g. predict 'not taken'); if the prediction is wrong, flush the incorrectly fetched instructions and restart fetch from the correct target.
- Step 6: Record the number of stall/flush cycles introduced in each case.

Observation Table

Hazard Type	Cause	Resolution Technique	Stall Cycles Saved
Structural	Shared memory port for IF & MEM	Separate instruction/data memory or extra port	1 cycle/instr
Data (RAW)	SUB depends on ADD result	Data Forwarding	2 cycles/instr
Control	Branch outcome unknown at fetch	Branch Prediction	1–2 cycles/branch

Result / Analysis

Without any hazard-handling, the dependent SUB and AND instructions each required 2 stall cycles to wait for the register write-back, giving 4 bubble cycles overall. Introducing data forwarding eliminated these stalls completely by routing the ALU result directly to the next instruction's execute stage. Branch prediction similarly avoided stalling the pipeline on every branch, at the cost of a flush penalty only on a misprediction. This confirms that hazard-resolution techniques are essential to keep pipeline throughput close to its ideal value of one instruction per cycle.

EXPERIMENT 5: Superscalar, Out-of-Order, Speculative Execution & SMT

Aim

To analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and to extend the study to hyper-threading and simultaneous multithreading (SMT), evaluating instruction throughput and resource utilization.

Tools Used

- Python (issue-width / IPC calculator)
- gem5 / Tomasulo simulator (conceptual reference)

Theory

- Superscalar Execution: the processor has multiple parallel execution units and can fetch, decode and issue more than one instruction per clock cycle (issue width > 1).

- Out-of-Order Execution: instructions are executed as soon as their operands are ready rather than strictly in program order, using techniques such as Tomasulo's algorithm and reservation stations, then results are committed in order (in-order retirement).
- Speculative Execution: the processor predicts the outcome of branches (or other uncertain events) and executes instructions beyond them speculatively; correct predictions improve throughput while mispredictions require squashing speculative results.
- Simultaneous Multithreading (SMT) / Hyper-Threading: multiple hardware threads share the same superscalar core each cycle, issuing instructions from different threads into idle execution-unit slots to raise overall resource utilization.

Algorithm

- Step 1: Define the issue width (number of instructions the processor can issue per cycle).
- Step 2: For a scalar (1-wide) processor, compute cycles = $n \text{ instructions} / 1$.
- Step 3: For a superscalar processor of width w , compute ideal cycles = n / w (assuming no dependency stalls).
- Step 4: Model out-of-order issue by allowing independent instructions to be scheduled ahead of a stalled instruction, using a small reservation-station queue.
- Step 5: Model speculative execution by predicting a branch and, on misprediction, discarding the speculatively issued instructions and recomputing the cycle count.
- Step 6: Model SMT by interleaving instructions from 2 threads into unused issue slots each cycle and recomputing effective utilization.
- Step 7: Compare instruction throughput (IPC) and functional-unit utilization across all configurations.

Observation Table

Configuration	Issue Width	Cycles (n=12)	IPC	Resource Utilization
Scalar (in-order)	1	12	1.00	Low
Superscalar (inorder)	4	3	4.00 (ideal)	Medium
Superscalar + OoO	4	2	6.00	High
Superscalar + OoO + SMT (2T)	4	2 (2 threads)	~6.00 per thread-slot	Highest

Result / Analysis

The scalar processor required 12 cycles to issue 12 instructions ($IPC = 1$). Widening the pipeline to a 4-issue superscalar design reduced this to an ideal 3 cycles, but real dependencies prevent the full width from being used every cycle. Allowing out-of-order issue let independent instructions fill slots that would otherwise stall, further reducing effective cycles and raising IPC. Adding SMT allowed a second thread to occupy execution slots left idle by the first thread's dependencies, increasing overall functional-unit utilization without requiring additional execution hardware. Together these techniques show a clear progression — superscalar increases raw issue capacity, out-of-order execution improves scheduling efficiency, speculative execution reduces control-hazard stalls, and SMT maximizes utilization of the available execution resources.

Code of Conduct:

I G Kushwanth(192424150) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G Kushwanth